Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 3

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Персистентность и изоляция: добавляем БД и настраиваем сети

Обучающийся Андреев Александр

Факультет Прикладной информатики

Группа К3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Андреев А. А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 ЗАПУСК КОНТЕЙНЕРА С ИМЕНОВАННЫМ VOLUME 4
1.1 Создание тома и запуск PostgreSQL 4
1.2 Подключение приложения и проверка API 5
2 ТЕСТ ПЕРСИСТЕНТНОСТИ: ПЕРЕСОЗДАНИЕ 6
3 BIND MOUNT ДЛЯ КОНФИГУРАЦИИ 7
3.1 Подключение файла с хоста 7
3.2 Изменение конфигурации без пересборки 8
4 ПОЛЬЗОВАТЕЛЬСКИЕ СЕТИ ДЛЯ ИЗОЛЯЦИИ 9
4.1 Перенос приложения и БД в общую сеть 9
4.2 Проверка API в пользовательской сети 10
4.3 Проверка разрешения имён и связности 11
5 ФИНАЛЬНЫЙ АРТЕФАКТ 12
ЗАКЛЮЧЕНИЕ 13

ВВЕДЕНИЕ

Цель работы – изучить постоянное хранение данных в Docker, подключить PostgreSQL к приложению, использовать именованные тома и Bind Mount, а также организовать и проверить пользовательскую сеть Docker.

Работа выполнена 03.11.2026 в Ubuntu-Lab. Персональные параметры приведены в таблице 1.

Таблица 1 – Персональные параметры

Параметр	**Значение**
Студент	Андреев Александр
Номер ИСУ	464998
Группа	К3320
Пользователь и хост	vboxuser@Ubuntu-Lab
Персональный префикс	andreev
Порт PostgreSQL на хосте	5528
Порт приложения	8098
Учебный секретный ключ	secret98andreev
Пользователь PostgreSQL	user_98
Учебный пароль PostgreSQL	pgpass_98
Имя базы данных	andreev98
Именованный том	andreev-pgdata
Пользовательская сеть	andreev-backend

Порты рассчитаны по последним двум цифрам ИСУ: 5430 + 98 = 5528 для PostgreSQL и 8000 + 98 = 8098 для приложения. Внутри контейнера PostgreSQL использует стандартный порт 5432.

1 ЗАПУСК КОНТЕЙНЕРА С ИМЕНОВАННЫМ VOLUME

1.1 Создание тома и запуск PostgreSQL

Docker Volume – хранилище, которым управляет Docker отдельно от файловой системы контейнера. В работе том andreev-pgdata подключён к каталогу данных PostgreSQL. Удаление контейнера само по себе не удаляет этот именованный том.

```
docker volume create andreev-pgdata
```

Контейнер PostgreSQL 15 запущен с персональными параметрами. Порт 5528 Ubuntu направлен на порт 5432 контейнера:

```
docker run -d \
  --name andreev-postgres \
  -e POSTGRES_USER=user_98 \
  -e POSTGRES_PASSWORD=pgpass_98 \
  -e POSTGRES_DB=andreev98 \
  -v andreev-pgdata:/var/lib/postgresql/data \
  -p 5528:5432 \
  postgres:15
docker exec andreev-postgres pg_isready -U user_98 -d andreev98
```

Проверка pg_isready завершилась успешно: сервер готов принимать подключения. Данные базы размещаются в andreev-pgdata, поэтому их жизненный цикл не зависит от отдельного контейнера PostgreSQL.

В приложение добавлены database.py для подключения и сессий SQLAlchemy, models.py с моделью таблицы notes и crud.py с операциями записи и чтения. В requirements.txt включены SQLAlchemy 2.0.35 и psycopg2-binary 2.9.10. В ЛР2 заметки хранились в памяти Python, теперь они записываются в PostgreSQL.

1.2 Подключение приложения и проверка API

Образ пересобран как andreev-assistant:v3. Dockerfile запускает Uvicorn на 0.0.0.0:8098. На первом этапе приложение обращается к опубликованному порту БД через хост Ubuntu:

```
docker build -t andreev-assistant:v3 .
docker run -d \
  --name andreev-assistant \
  -p 8098:8098 \
  -e APP_OWNER=andreev \
  -e APP_SECRET=secret98andreev \
  --add-host=host.docker.internal:host-gateway \
  -e DATABASE_URL="postgresql://user_98:pgpass_98@host.docker.internal:5528/andreev98" \
  andreev-assistant:v3
```

На рисунке 1 приведены четыре проверки: GET /, GET /health, POST /notes и GET /notes. Главная страница вернула приветствие владельцу andreev и версию 2.0; /health – status="healthy" и owner="andreev".

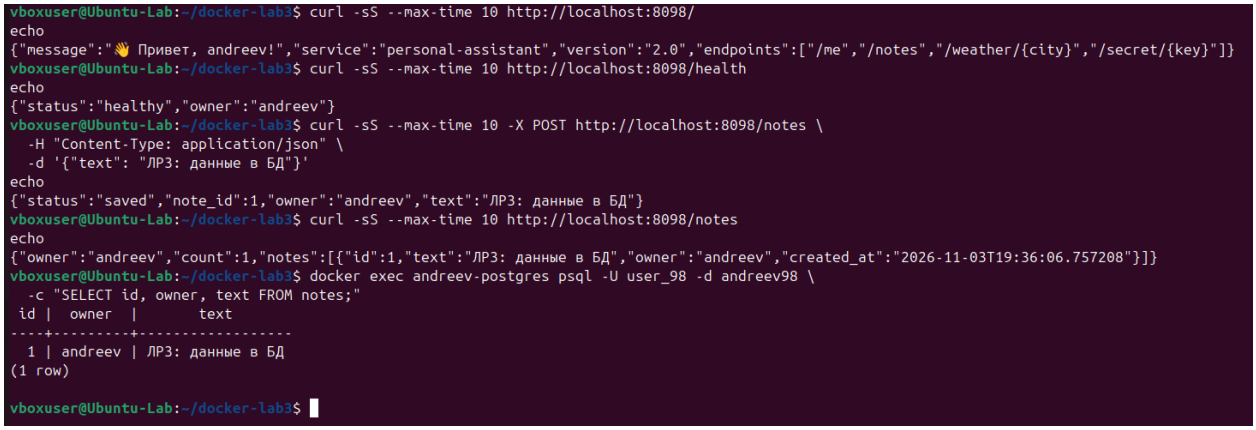


Рисунок 1 – Проверка API и записи в таблице notes

Создана заметка «ЛР3: данные в БД» с note_id=1. GET /notes вернул count=1. SQL-запрос SELECT id, owner, text FROM notes; также показал одну запись с id=1 и владельцем andreev.

Маршрут /health в исходном коде не выполняет запрос к базе. Работу PostgreSQL подтверждают успешные запись и чтение заметки, а также результат SQL-запроса на рисунке 1.

2 ТЕСТ ПЕРСИСТЕНТНОСТИ: ПЕРЕСОЗДАНИЕ

Персистентность – сохранение данных после завершения или пересоздания процесса либо контейнера. Для проверки сначала получили список заметок, затем остановили и удалили контейнер приложения:

```
curl -sS http://localhost:8098/notes
docker stop andreev-assistant
docker rm andreev-assistant
```

После этого создан новый andreev-assistant из прежнего образа andreev-assistant:v3 с той же командой запуска и DATABASE_URL, что в разделе 1.2. Контейнер PostgreSQL и том andreev-pgdata на этом этапе не удалялись. Затем повторно выполнен GET /notes.

Артефакт проверки на рисунке 2 содержит дату 03.11.2026, время 19:43:21 и пользователя vboxuser@Ubuntu-Lab. Получен count=1; сохранены id=1, владелец andreev и текст «ЛР3: данные в БД». Русский текст в JSON представлен Unicode-последовательностями.

```
vboxuser@Ubuntu-Lab:~/docker-lab3$ echo "======================================" && \
echo "Тест персистентности: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "======================================" && \
curl -sS http://localhost:8098/notes | python3 -m json.tool
======================================
Тест персистентности: 03.11.2026 19:43:21
Студент: vboxuser@Ubuntu-Lab
======================================
{
    "owner": "andreev",
    "count": 1,
    "notes": [
        {
            "id": 1,
            "text": "\u041b\u04203: \u0434\u0430\u043d\u043d\u044b\u0435 \u0432 \u0411\u0414",
            "owner": "andreev",
            "created_at": "2026-11-03T19:36:06.757208"
        }
    ]
}
vboxuser@Ubuntu-Lab:~/docker-lab3$
```

Рисунок 2 – Заметка после пересоздания контейнера приложения

Заметка осталась доступна, поскольку хранится в PostgreSQL, а не в памяти или файловом слое приложения. Новый контейнер подключился к той же базе. Этот этап проверяет пересоздание приложения; пересоздание самой БД с прежним томом выполнено далее, на этапе настройки сети.

3 BIND MOUNT ДЛЯ КОНФИГУРАЦИИ

3.1 Подключение файла с хоста

Bind Mount подключает существующий файл или каталог хоста непосредственно в файловую систему контейнера. На Ubuntu создан каталог config с файлом app.conf:

```
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=false
owner=andreev
```

Чтобы освободить порт 8098, контейнер andreev-assistant остановлен и удалён. Затем приложение запущено как andreev-assistant-config с тем же подключением к БД и дополнительным монтированием каталога:

```
docker stop andreev-assistant
docker rm andreev-assistant
docker run -d --name andreev-assistant-config \
  -p 8098:8098 \
  -e APP_OWNER=andreev -e APP_SECRET=secret98andreev \
  --add-host=host.docker.internal:host-gateway \
  -e DATABASE_URL="postgresql://user_98:pgpass_98@host.docker.internal:5528/andreev98" \
  -v "$(pwd)/config:/app/config:ro" \
  andreev-assistant:v3
```

Здесь $(pwd)/config – каталог на хосте, /app/config – точка подключения внутри контейнера, а ro запрещает запись в этот каталог из контейнера. Файл на Ubuntu остаётся доступным для изменения.

Через docker exec -it andreev-assistant-config sh открыта оболочка контейнера. Команда cat /app/config/app.conf показала исходную конфигурацию и owner=andreev (рисунок 3).

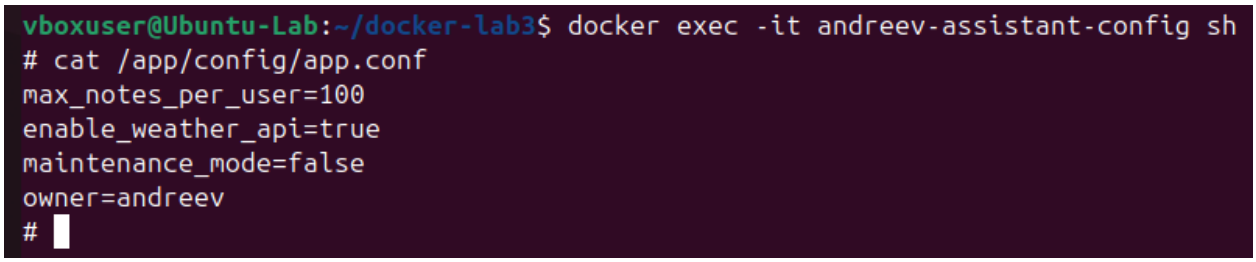


Рисунок 3 – Исходный app.conf внутри контейнера

3.2 Изменение конфигурации без пересборки

На хосте в config/app.conf значение maintenance_mode=false изменено на maintenance_mode=true. Затем выполнены перезапуск контейнера и повторное чтение файла:

```
docker restart andreev-assistant-config
docker exec andreev-assistant-config cat /app/config/app.conf
```

На рисунке 4 видно обновлённое значение maintenance_mode=true. Остальные параметры, включая owner=andreev, сохранились. Пересборка Docker-образа для этого не потребовалась.

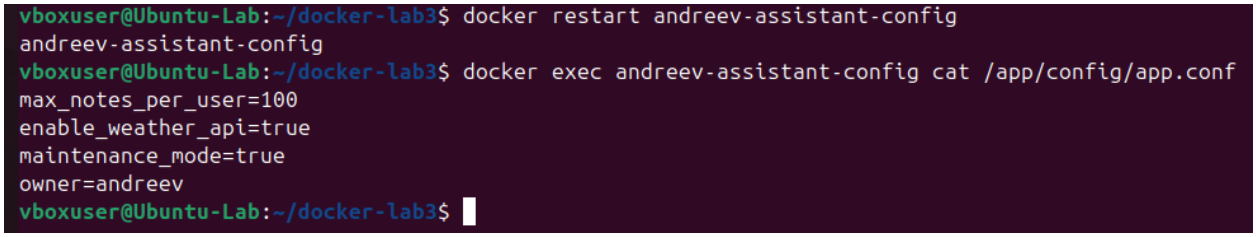


Рисунок 4 – Обновлённая конфигурация после docker restart

Эксперимент показывает, что контейнер видит изменения файла хоста через Bind Mount. В опубликованной конфигурации также сохранено maintenance_mode=true. Полноценная логика режима обслуживания этим опытом не проверялась: приложение не читает этот параметр в обработчиках API.

Docker Volume подходит для данных PostgreSQL: Docker управляет расположением хранилища, а контейнер получает доступ к нему по имени тома. Bind Mount связан с конкретным путём хоста и удобен для конфигурации, которую нужно редактировать снаружи.

В обоих случаях подключённые данные находятся вне записываемого слоя контейнера. Однако при Bind Mount перенос проекта на другой хост требует наличия соответствующего каталога. Для PostgreSQL важно повторно подключать именно прежний том andreev-pgdata.

4 ПОЛЬЗОВАТЕЛЬСКИЕ СЕТИ ДЛЯ ИЗОЛЯЦИИ

4.1 Перенос приложения и БД в общую сеть

Создана пользовательская сеть andreev-backend с драйвером bridge. Старые контейнеры остановлены и удалены, после чего PostgreSQL запущен в новой сети с прежним томом:

```
docker network create andreev-backend
docker stop andreev-postgres andreev-assistant-config
docker rm andreev-postgres andreev-assistant-config
docker run -d --name andreev-postgres \
  --network andreev-backend \
  -e POSTGRES_USER=user_98 \
  -e POSTGRES_PASSWORD=pgpass_98 \
  -e POSTGRES_DB=andreev98 \
  -v andreev-pgdata:/var/lib/postgresql/data \
  postgres:15
```

В команде PostgreSQL больше нет -p: порт 5432 не публикуется на Ubuntu. Приложение подключено к andreev-backend и использует адрес andreev-postgres:5432. Каталог config подключён через Bind Mount, наружу опубликован только порт приложения 8098.

Ранее host.docker.internal указывал на хост, через который был доступен опубликованный порт 5528. В Ubuntu это имя добавлялось флагом --add-host=host.docker.internal:host-gateway. В общей пользовательской сети Docker разрешает имя andreev-postgres, поэтому приложению достаточно имени контейнера и внутреннего порта 5432.

Команда запуска приложения и ответы API приведены на рисунке 5. После пересоздания PostgreSQL прежняя заметка сохранилась. POST /notes добавил «ЛР3: сеть работает!» с note_id=2, а GET /notes вернул обе заметки и count=2.

4.2 Проверка API в пользовательской сети

Маршрут /health вернул healthy. Чтение прежней заметки и создание новой подтвердили обмен приложения с PostgreSQL через andreev-backend (рисунок 5).

```
vboxuser@Ubuntu-Lab:~/docker-lab3$ docker run -d \
  --name andreev-assistant \
  --network andreev-backend \
  -p 8098:8098 \
  -e APP_OWNER=andreev \
  -e APP_SECRET=secret98andreev \
  -e DATABASE_URL="postgresql://user_98:pgpass_98@andreev-postgres:5432/andreev98" \
  -v "$(pwd)/config:/app/config:ro" \
  andreev-assistant:v3
9d983168d40780ddb07216fc8f0d7889669c8bfbb9972b803fb0db4336eb61b2
vboxuser@Ubuntu-Lab:~/docker-lab3$ curl -sS http://localhost:8098/health
echo
{"status":"healthy","owner":"andreev"}
vboxuser@Ubuntu-Lab:~/docker-lab3$ curl -sS http://localhost:8098/notes
echo
{"owner":"andreev","count":1,"notes":[{"id":1,"text":"ЛР3: данные в БД","owner":"andreev","created_at":"2026-11-03T19:36:06.757208"}]}
vboxuser@Ubuntu-Lab:~/docker-lab3$ curl -sS -X POST http://localhost:8098/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР3: сеть работает!"}'
echo
{"status":"saved","note_id":2,"owner":"andreev","text":"ЛР3: сеть работает!"}
vboxuser@Ubuntu-Lab:~/docker-lab3$ curl -sS http://localhost:8098/notes
echo
{"owner":"andreev","count":2,"notes":[{"id":2,"text":"ЛР3: сеть работает!","owner":"andreev","created_at":"2026-11-03T19:56:26.956542"},{"id":1,"text":"ЛР3: данные в БД","owner":"andre
ev","created_at":"2026-11-03T19:36:06.757208"}]}
vboxuser@Ubuntu-Lab:~/docker-lab3$
```

Рисунок 5 – Запуск приложения в сети andreev-backend и две сохранённые заметки

Первый GET /notes содержит count=1, последующий – count=2. Данные первоначальной заметки сохранились после пересоздания контейнера PostgreSQL с подключением andreev-pgdata.

4.3 Проверка разрешения имён и связности

Для проверки использован временный Alpine-контейнер. Сначала он запущен вне andreev-backend, затем – с подключением к этой сети:

```
docker run --rm alpine:3.18 ping -c 2 andreev-postgres
docker run --rm --network andreev-backend \
  alpine:3.18 ping -c 2 andreev-postgres
docker network inspect andreev-backend \
  --format '{{range .Containers}}{{.Name}} {{end}}'
```

В первом случае получено ping: bad address 'andreev-postgres'. Во втором имя разрешилось в 172.18.0.2; получены два ответа: 2 packets transmitted, 2 packets received, 0% packet loss. Результаты показаны на рисунке 6.

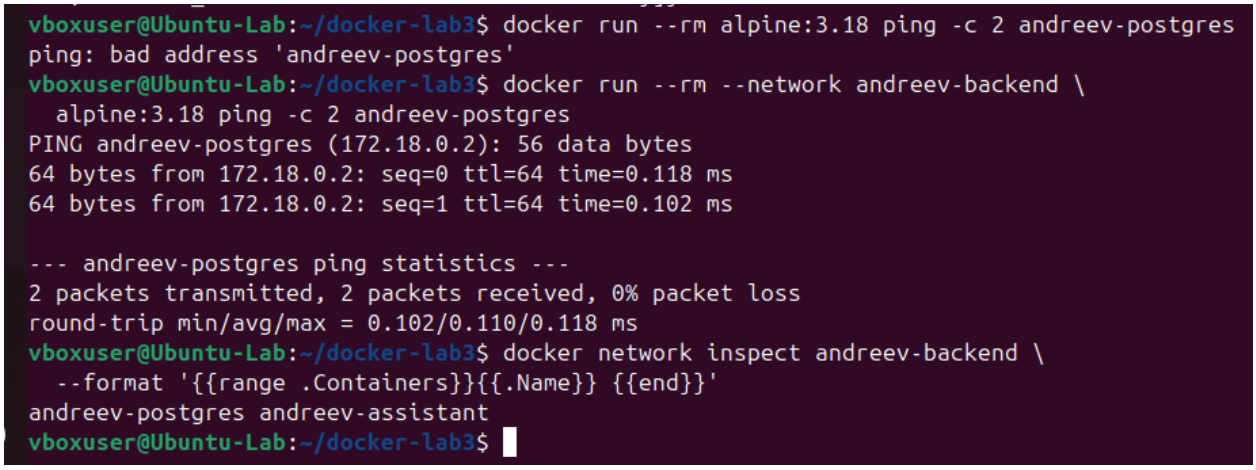


Рисунок 6 – Проверка имени PostgreSQL вне сети и внутри неё

Вывод docker network inspect подтвердил присутствие andreev-postgres и andreev-assistant. Оба контейнера находятся в одной пользовательской сети.

Опыт подтверждает разрешение имени контейнера и успешную ICMP-связь внутри andreev-backend. Ошибка имени вне этой сети сама по себе не доказывает абсолютную недоступность PostgreSQL извне: для такого вывода потребовались бы дополнительные проверки маршрутизации и правил доступа.

5 ФИНАЛЬНЫЙ АРТЕФАКТ

Финальная команда вывела дату и время, пользователя и хост, работающие контейнеры, именованные тома и пользовательские сети. Полный вывод представлен на рисунке 7.

```
vboxuser@Ubuntu-Lab:~/docker-lab3$ echo "======================================" && \
echo "ЛР3 завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "======================================" && \
echo "" && \
echo "Контейнеры:" && \
docker ps --filter "name=andreev-" \
  --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" && \
echo "" && \
echo "Volumes:" && \
docker volume ls --filter "name=andreev-" && \
echo "" && \
echo "Сети:" && \
docker network ls --filter "name=andreev-"
======================================
ЛР3 завершена: 03.11.2026 20:01:11
Студент: vboxuser@Ubuntu-Lab
======================================

Контейнеры:
NAMES               STATUS         PORTS
andreev-assistant   Up 5 minutes   0.0.0.0:8098->8098/tcp, [::]:8098->8098/tcp
andreev-postgres    Up 5 minutes   5432/tcp

Volumes:
DRIVER    VOLUME NAME
local     andreev-pgdata

Сети:
NETWORK ID     NAME              DRIVER    SCOPE
173baf5e9c8c   andreev-backend   bridge    local
vboxuser@Ubuntu-Lab:~/docker-lab3$
```

Рисунок 7 – Итоговое состояние инфраструктуры Docker

Проверка выполнена 03.11.2026 в 20:01:11 пользователем vboxuser@Ubuntu-Lab. Контейнеры andreev-assistant и andreev-postgres имеют статус Up 5 minutes. У приложения опубликован порт 8098→8098/tcp; у PostgreSQL указан только внутренний 5432/tcp.

Том andreev-pgdata использует драйвер local. Сеть andreev-backend имеет драйвер bridge и область local. Все необходимые компоненты присутствуют и согласованы с предыдущими проверками.

ЗАКЛЮЧЕНИЕ

В ходе работы приложение подключено к PostgreSQL, а данные базы вынесены в именованный том andreev-pgdata. Создание и получение заметок через API, а также SQL-запрос подтвердили запись данных в таблицу notes.

После удаления и пересоздания контейнера приложения заметка «ЛР3: данные в БД» сохранилась. При последующем пересоздании PostgreSQL в пользовательской сети был подключён тот же том, поэтому первая заметка осталась доступной и после этого этапа.

С помощью Bind Mount каталог config подключён в режиме только для чтения. Изменение maintenance_mode на хосте стало видно внутри контейнера после перезапуска без пересборки образа. Проверено содержимое файла; работа режима обслуживания приложения не исследовалась.

Создана сеть andreev-backend. Приложение взаимодействует с БД по имени andreev-postgres без публикации порта PostgreSQL на хосте. Проверки Alpine показали ошибку разрешения имени вне сети и два успешных ответа внутри неё. Через API добавлена вторая заметка «ЛР3: сеть работает!».

Освоены постоянное хранение данных, подключение конфигурации с хоста и организация сетевого взаимодействия контейнеров. Финальный артефакт подтвердил наличие двух работающих контейнеров, именованного тома и пользовательской сети.

Исходники и отчёт опубликованы в ветке lab-3:

https://github.com/alx-an/docker-lab2/tree/lab-3

Код расположен в solutions/assist-py/, отчёт – в solutions/reports/lab-3/. Команды запуска выполняются из solutions/assist-py/. На исходных скриншотах использован рабочий каталог ~/docker-lab3; перенос файлов в структуру репозитория не изменяет параметры контейнеров.